

Name: - Aditya Ganguly

Reg.No.: - 25BCE10661

Branch: - CSE Core

Faculty:-Dr. Rakesh Srivastava

VIT Bhopal

Project Report

Title:- Spam-Email-Filtering-Machine

Introduction:-

The objective of this project is to build a supervised machine learning model capable of distinguishing between legitimate emails ("Ham") and unsolicited bulk emails ("Spam"). As communication volume increases, automated filtering becomes essential for security and productivity.

Problem Statement:-

The E-Mails remain full of spam mails and thus cannot be sorted out easily causing important mails skipped by the user. Text data is unstructured and cannot be processed directly by mathematical models. The challenge lies in:

1. Converting human language into numerical features.
2. Training a classifier that can handle the frequency of specific "trigger words."
3. Ensuring the dataset is balanced and correctly aligned to avoid training errors.

Proposed solution:-

The solution to sort the emails or find whether an email is fraud/spam or not can be done by this machine. The machine has been trained with approx. 100+ unique email types and thus has been assigned as 0 for not a spam email and 1 for a spam email. Thus here the human language has been converted in terms of number form in 0 and 1 form. The model has been trained such that it can identify the triggered words in the email and mark it as spam.

Methodology

➤ Data Preparation:-

The dataset consists of **100 unique email subject lines**.

- **Label 0 (Not Spam):** Work-related, personal, or transactional subjects (e.g., "Meeting at 2 PM").
- **Label 1 (Spam):** Promotional, urgent, or fraudulent subjects (e.g., "Win a Million Dollars").

To ensure model stability, the code includes a synchronization step:

```
min_length = min(len(emails), len(labels))
```

This prevents ValueError during the data splitting phase.

➤ Feature Engineering (Vectorization)

We employed the **CountVectorizer** (Bag of Words) approach. This technique creates a vocabulary of all unique tokens across the dataset and represents each email as a vector of word frequencies.

➤ Algorithm Selection

The **Multinomial Naive Bayes (MNB)** classifier was selected. MNB is based on Bayes' Theorem:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

It is particularly well-suited for text classification because it assumes that the presence of one word is independent of the presence of another, making it computationally efficient and accurate for high-dimensional data.

Implementation Steps

1. **Data Loading:** Inputting the raw text and binary labels.
2. **Preprocessing:** Tokenizing the strings and removing casing sensitivity.
3. **Splitting:** Dividing the data into **80% Training** and **20% Testing** sets to validate the model on unseen data.
4. **Training:** Fitting the MNB model to the training vectors.
5. **Testing:** Evaluating the model against the test set.

Results and Observations

- **Accuracy:** The model typically achieves high accuracy (85-95%) on this controlled dataset.
- **Key Indicators:** The model successfully identifies "Spam" based on high-frequency tokens like *Free*, *Win*, *Urgent*, and *Account*.
- **Limitations:** Since the model only looks at subject lines, it may struggle with "Short Text" ambiguity (e.g., a short personal email that looks like a notification).

Challenges faced:-

The first and foremost challenge that I faced was that the scikit-learn module was not being applied to python. But this issue simply got resolved after restarting my device.

The next challenge was to train the machine model by giving it a 100+ email types and assigning a data set like 0 or 1 as per the email. This consumed a lot of time but after all I could manage to do it perfectly with some patience.

After completing the code, I was getting a minor track_steps error which I resolved quickly by generating help from ai to find my mistake.

Goal of the program:-

The main goal of this program was to find the spam email for each new mail that arrives and indicate whether it is a spam mail or not.

What I learnt from this Project:-

While doing the project of spam email filtering machine, I learn how to train an AI model and then also implement it in the program. I also learnt that certain problems may arise but to not get scared and keep finding its solution until found.

Future Scope

While the current model provides a solid foundation for spam detection, the following enhancements would significantly improve its robustness and real-world utility:

➤ **Advanced Natural Language Processing (NLP)**

Currently, the model treats every word as a unique entity. Future versions will implement:

- **Stop-word Removal:** Filtering out common words like "the," "is," and "and" that do not contribute to the "spamminess" of an email.
- **Lemmatization/Stemming:** Reducing words to their root form (e.g., "winning" and "wins" both become "win") to consolidate the feature set.
- **TF-IDF Vectorization:** Moving beyond simple word counts to **Term Frequency-Inverse Document Frequency**, which weighs unique, meaningful words more heavily than common ones.

➤ **Integration of Deep Learning**

To handle more complex sentence structures and "sneaky" spam (like leetspeak or intentional misspellings), we can transition from Naive Bayes to:

- **Recurrent Neural Networks (RNNs) or LSTMs**, which understand the *sequence* and context of words rather than just treating them as a "bag of words."
- **Transformer Models (BERT):** Using pre-trained models to understand the semantic meaning behind subject lines.

➤ **Real-time Deployment & UI**

The model currently runs in a static Python environment. Future development includes:

- **Web Interface:** Building a user-friendly dashboard using **Streamlit** or **Flask** where users can paste an email subject and get an instant "Spam/Ham" prediction.
- **API Integration:** Developing a REST API so that third-party email clients can send subjects to this model for real-time filtering.

➤ **Metadata Analysis**

Spam detection is often more accurate when combined with non-text data. Future scopes include analyzing:

- **Sender Reputation:** Checking the sender's IP or domain against global blacklists.
- **Time-of-Day Analysis:** Identifying patterns in when bulk spam is typically sent.

Conclusion

The project successfully demonstrates that even a simple Naive Bayes model can effectively filter spam when provided with a clean, synchronized dataset. Future iterations could include **Stop-word removal** (filtering out "the", "a", "is") and **Lemmatization** to further refine the feature set.